

Base de Conocimiento Vectorial

Informe Técnico — Etapa 1

CODEFEST AD ASTRA 2026 · Fuerza Aeroespacial Colombiana · Universidad de los Andes

1. Resumen

Construimos un sistema de **recuperación densa multilingüe** que indexa un corpus multiformato (ES/EN/PT) sobre los tres fenómenos del reto y responde consultas en lenguaje natural devolviendo los 3 documentos y los 10 fragmentos más relevantes.

El diseño parte de una lectura precisa de las reglas de evaluación (Sección 10.2.1 de la especificación): **la relevancia de un fragmento se juzga sobre su contenido textual (text) y la de un documento sobre el campo fuente**, no sobre los identificadores internos que asigna cada equipo. De ahí nuestra prioridad de ingeniería: la *fidelidad de la extracción y la limpieza del texto* determinan el techo de las métricas, por encima de cualquier ajuste fino del recuperador.

Extracción → Limpieza → Chunking (2 niveles) → Encoder(s) → FAISS
 → Fusión RRF → [Reranking] → Agregación chunk→doc → resultados.jsonl
 (+ Grafo de conocimiento)

2. Preprocesamiento: extracción y limpieza

2.1 Extracción por formato

Formato	Herramienta	Motivo
PDF	Docling (MIT) + PyMuPDF	Docling reconstruye layout, tablas y orden de lectura; PyMuPDF garantiza robustez si Docling falla
HTML	trafilatura	Elimina <i>boilerplate</i> conservando el cuerpo del artículo
JSON	selección de campos	Concatena <code>title/body_text</code> en orden; <code>url, date</code> van a metadata
CSV/XLSX	pandas + openpyxl	Cada fila serializada como pares <code>columna: valor</code> para preservar contexto
Imágenes	OCR multilingüe	Recupera texto de infografías y figuras
PBF	recorrido de capas	Atributos como pares clave-valor; una versión por elemento (evita duplicar por zoom)

La elección de Docling se validó empíricamente: sobre un PDF institucional real de 24 páginas extrajo **13 % más contenido** que PyMuPDF y fue el único que recuperó una tabla como estructura tabular en lugar de texto aplanado.

2.2 Limpieza y normalización

Esta etapa resultó la de **mayor impacto medido** del preprocesamiento. Al analizar la extracción de un PDF real detectamos tres patrones de ruido que degradaban el índice:

- Índices con puntos de relleno** (Introducción . . . 3): el segmentador interpretaba cada punto como fin de oración.
- Encabezados y pies repetidos** en cada página, que generaban fragmentos sin valor informativo y diluían la señal semántica.
- Guiones de corte de línea** (conges-\n tión), que parten palabras y rompen el emparejamiento léxico y semántico.

Nuestra limpieza normaliza a NFC, elimina caracteres de control, descarta líneas de índice, detecta *boilerplate* por repetición y **re-une palabras cortadas por guion**, distinguiéndolas de compuestos legítimos (*space-debris* no se altera).

Efecto medido: en los primeros 3 000 caracteres del documento de prueba, las falsas oraciones detectadas pasaron de **279 a 59**, y desaparecieron los fragmentos compuestos únicamente por el índice del documento.

3. Estrategia de chunking

3.1 Chunking de dos niveles

La especificación impone dos restricciones que operan en momentos distintos: la **completitud lingüística** (Sección 3.3), que prohíbe fragmentos con oraciones incompletas, y el **límite de 250 palabras** por fragmento de salida (Sección 9.2). El tamaño óptimo para *codificar* no coincide con el máximo de *presentación*, así que los separamos:

- **Nivel 1 — chunk de índice.** Agrupa oraciones completas hasta un máximo configurable de tokens (256–384, dentro del límite del encoder), con solapamiento de una oración entre fragmentos consecutivos. Es la unidad que se codifica y almacena en FAISS.
- **Nivel 2 — sub-fragmento de salida.** Al construir la respuesta, un chunk que exceda las 250 palabras se divide respetando de nuevo los límites oracionales. Los sub-fragmentos conservan el `chunk_id` original, que cumple función de **trazabilidad** hacia la evidencia indexada.

3.2 Justificación

- *Frente a tamaño fijo de tokens:* cortar por conteo parte oraciones y viola el requisito obligatorio de la Sección 3.3.
- *Frente a chunking por párrafo puro:* los párrafos de documentos técnicos van de una línea a más de una página; produce fragmentos desiguales que pueden superar la ventana del encoder.
- *A favor del solapamiento:* una idea en la frontera entre dos fragmentos quedaría representada a medias en ambos; solapar una oración lo mitiga con bajo coste.

La segmentación oracional usa **pysbd** (multilingüe), con un segmentador de respaldo basado en reglas para que el sistema opere aunque falte la dependencia.

3.3 Metadata por fragmento

Cada chunk almacena los ocho campos obligatorios de la Tabla 1 más campos opcionales (`idioma`, `título`, `fecha`). El campo `fuelle` se normaliza siempre con separadores `/`: durante el desarrollo detectamos que en Windows se guardaba con `\`, lo que **anulaba el emparejamiento a nivel documento** y llevaba el F1@3 a cero pese a una recuperación correcta.

4. Codificación semántica: selección de encoder

4.1 Encoder principal: BAAI/bge-m3

Criterio (Sección 4.3)	Cumplimiento
Soporte multilingüe	Nativo en ES/EN/PT, entrenado para recuperación cross-lingual
Dimensionalidad	1024 — equilibrio entre expresividad y coste
Longitud de entrada	Hasta 8192 tokens, muy por encima de los 512 habituales
Benchmarks	Estado del arte en MIRACL/MTEB multilingüe para recuperación densa
Licencia	MIT
Eficiencia	Ejecuta en una GPU de 8 GB en fp16

El criterio decisivo es el **cross-lingual**: el conjunto de evaluación reparte las consultas entre español, inglés y portugués, y una consulta en un idioma debe recuperar documentos en los otros dos. Un encoder monolingüe fallaría en dos tercios de las consultas.

Validación empírica. BGE-M3 obtuvo NDCG@10 de **0,990** frente a **0,966** de **paraphrase-multilingual-MiniLM-L12-v2**, con la diferencia concentrada exactamente donde se predijo: la consulta en portugués pasó de 0,877 a **1,000**.

4.2 Restricción de arquitectura

Todos los modelos empleados en indexación y recuperación son **encoders** (familia BERT). No se utiliza ningún modelo generativo (decoder) en la construcción del índice ni en la recuperación, conforme a las Secciones 4.2 y 8.3.

5. Índice vectorial FAISS

Elegimos **IndexFlatIP con vectores normalizados a norma unitaria**, que equivale exactamente a similitud coseno (ecuación 4 de la especificación).

Justificación: para el volumen de este reto la búsqueda exhaustiva es holgadamente viable y ofrece resultados **exactos**. Los índices aproximados (IVFFlat, HNSW) sacrifican exactitud por velocidad, un intercambio que no compensa cuando la métrica evaluada es la calidad del ranking y el tiempo de respuesta no se puntúa. Si el corpus creciera un orden de magnitud, migrar a **IndexHNSWFlat** es un cambio de una línea en la configuración.

FAISS almacena únicamente vectores e identificadores internos. La metadata vive en `metadata.jsonl`, donde la línea *i* **corresponde al identificador interno *i*** que FAISS asigna al indexar, tal como exige el formato de entrega. La persistencia usa las funciones nativas `write_index/read_index`.

6. Módulo de recuperación

6.1 Fusión de múltiples índices

Implementamos las tres estrategias de la Sección 8.4 y adoptamos **Reciprocal Rank Fusion** por defecto:

$$s_{\text{RRF}}(c) = \sum_{j=1}^m \frac{1}{k_0 + r_j(c)}, \quad k_0 = 60$$

RRF opera sobre **posiciones** y no sobre puntuaciones, por lo que es robusto frente a las distintas escalas de similitud de encoders diferentes —un problema real, ya que CombSUM permitiría que el encoder con puntuaciones sistemáticamente más altas domine la fusión.

6.2 Agregación de fragmentos a documentos

Para el nivel documento (F1@3) agrupamos los chunks por `doc_id` y calculamos una puntuación agregada. Implementamos tres estrategias —máximo (*max pooling*), suma y media ponderada— comparadas empíricamente en la Sección 8.

6.3 Reranking con cross-encoder

Incluimos un paso opcional de reordenamiento con BAAI/bge-reranker-v2-m3, un **cross-encoder** que puntúa conjuntamente el par (consulta, fragmento).

Consideración sobre el reglamento. La Sección 8.3 prohíbe modelos **generativos** (decoders) en la recuperación. Un cross-encoder es arquitectura **encoder** (familia BERT) y produce una puntuación escalar de relevancia; no genera texto. Entendemos que su uso es admisible, pero por prudencia el paso es **desactivable por configuración** y, ante una interpretación restrictiva, el sistema opera con fusión RRF pura sin modificar una línea de código.

6.4 Cumplimiento del formato de salida

Un validador estricto verifica cada objeto antes de escribir `resultados.jsonl`: exactamente 3 documentos, exactamente 10 fragmentos, ningún fragmento por encima de 250 palabras y orden q001–q050. La verificación es automática y forma parte del pipeline de generación.

7. Grafo de conocimiento (componente bonus)

Construcción. (i) *Reconocimiento de entidades* con GLiNER multilingüe (MIT), modelo *zero-shot* que extrae tipos definidos en configuración sin reentrenamiento; verificamos su comportamiento cross-lingual: sobre texto en español identificó *Estados Unidos* (país), *sistemas de armas autónomas* (tecnología) y *Convenio de Ginebra* (evento). (ii) *Extracción de relaciones* por co-ocurrencia dentro del mismo fragmento, con peso acumulado; **no se emplea ningún modelo generativo**. (iii) *Persistencia* en grafo dirigido NetworkX exportado a `grafo.graphml`, donde cada nodo guarda los `chunk_id` en que aparece la entidad y cada arista referencia el `doc_id` y `chunk_id` que la respalda, garantizando **trazabilidad** de toda relación hasta su evidencia textual.

Integración con la recuperación. El grafo actúa como **índice adicional** dentro del esquema de fusión (Sección 8.5): se extraen las entidades de la consulta con el mismo NER, se recuperan los chunks vinculados a esas entidades y a sus **vecinos de primer orden**, se puntúan por evidencia acumulada y el ranking resultante se fusiona con los vectoriales mediante RRF. Aporta una señal que los embeddings sólo capturan de forma implícita: **relaciones explícitas entre entidades**.

8. Metodología de evaluación y resultados

8.1 Optimizar sin ground truth

El conjunto de evaluación y sus juicios de relevancia no son públicos durante el reto. Ajustar hiperparámetros «a ojo» equivale a no ajustarlos. Construimos un **arnés de evaluación interno**: (i) consultas propias con relevancia conocida, equilibradas entre fenómenos e idiomas; (ii) una implementación de **NDCG@10 y F1@3 idéntica a la especificación** (ecuaciones 8–14), verificada con pruebas unitarias que reproducen el ejemplo de Conteo de Borda de la Tabla 3; (iii) un barrido que rankea cada configuración con **Conteo de Borda**, el mismo criterio del leaderboard oficial, optimizando así el equilibrio entre ambas métricas y no una sola.

Este arnés detectó dos defectos que habrían costado puntos y que ninguna inspección visual habría revelado: el separador de rutas en **fuentes** y un error en el ranking ideal del NDCG que producía valores superiores a 1.

8.2 Corpus de validación

Para validar en condiciones realistas antes de disponer del corpus oficial, reunimos **50 documentos públicos auténticos** (111 MB): artículos de arXiv sobre los tres fenómenos, el *ESA Space Environment Report*, el informe IADC de UNOOSA y el *Panorama Social* de la CEPAL. El conjunto reproduce las dificultades del escenario real: documentos **largos** (el informe de la CEPAL supera los 980 000 caracteres y genera 654 fragmentos por sí solo), **multilingües** y con estructura académica e institucional real.

Juicios de relevancia. Aprovechamos una señal existente e independiente: el buscador de arXiv. Si una consulta devolvió un artículo, ese artículo es relevante para la consulta en lenguaje natural equivalente, y su *posición* da el grado ($1-2 \rightarrow 3$; $3-4 \rightarrow 2$; resto $\rightarrow 1$). Es supervisión débil, pero externa a nuestro sistema, lo que evita el sesgo circular de evaluarnos con juicios propios. Las consultas se redactaron deliberadamente en idiomas distintos al de los documentos para forzar el escenario cross-lingual.

8.3 Resultados

Los 50 documentos se procesaron **sin una sola omisión**, produciendo **2 601 fragmentos**. El almacén de metadata pasó la validación de esquema con **cero errores** y mantiene la correspondencia exacta línea $\leftrightarrow$ identificador interno de FAISS.

Configuración	NDCG@10	F1@3	Borda
max pooling + reranking	0,4246	0,5000	8
media ponderada	0,3625	0,5000	6
max pooling	0,3625	0,5000	6
media ponderada + reranking	0,4246	0,4697	5
suma	0,3625	0,4697	3

Hallazgo 1 — el reranking es la palanca más rentable. El cross-encoder eleva el NDCG@10 de 0,3625 a **0,4246**, una mejora relativa del **17 %**, sin degradar el F1@3. Es la mayor ganancia individual medida en todo el sistema.

Hallazgo 2 — los corpus de juguete engañan. Sobre un conjunto sintético reducido, las mismas variantes indicaban que el reranking **no aportaba nada** e incluso perjudicaba, porque con pocos documentos el recuperador denso ya saturaba las métricas ($\text{NDCG} \approx 0,95$ en toda variante). De haber confiado en ese conjunto habríamos descartado el componente más valioso.

Hallazgo 3 — la agregación por suma perjudica el nivel documento. En ambos corpus, sumar las puntuaciones de todos los fragmentos de un documento reduce el F1@3: favorece documentos largos con muchos fragmentos mediocres sobre documentos concisos con un fragmento excelente. Adoptamos **max pooling**.

Interpretación de las cifras absolutas. Los juicios de este conjunto son **incompletos por construcción**: cada consulta tiene unos cinco documentos etiquetados, pero el corpus contiene otros genuinamente relacionados que no lo están; cuando el sistema los recupera —acertadamente— computan como error y deprimen el NDCG. Estas cifras son válidas para **comparar configuraciones entre sí**, que es el uso que les damos, pero no son una estimación del rendimiento absoluto sobre el conjunto oficial, cuyos juicios sí son exhaustivos.

9. Reproducibilidad

El script `generador.py` reconstruye `resultados.jsonl` a partir del índice persistido y del archivo de consultas. El determinismo se asegura fijando las semillas de `random`, `NumPy` y `PyTorch`, activando algoritmos deterministas y escribiendo el JSON Lines con separadores fijos y codificación UTF-8 explícita. Toda la configuración —chunking, encoders, tipo de índice, fusión, agregación— reside en un único `config.yaml`, de modo que cualquier resultado reportado es rastreable a una configuración concreta. El proyecto cuenta con **42 pruebas automatizadas** que cubren las métricas, el validador de esquema, el chunker, las estrategias de fusión, la limpieza y el recuperador basado en grafo.

10. Conclusiones

La decisión de diseño más rentable no fue la elección del modelo, sino **leer la regla de emparejamiento y orientar el esfuerzo a la fidelidad del texto**. La limpieza del corpus produjo la mejora más barata; el encoder correcto (BGE-M3) resolvió el requisito cross-lingual; el reranking aportó la mayor ganancia medida (+17 % de NDCG@10); y el arnés de evaluación interno convirtió decisiones de intuición en decisiones medidas.

La lección metodológica que atraviesa el trabajo es que **medir en condiciones representativas cambia las conclusiones**. Dos hallazgos —el valor del reranking y el perjuicio de la agregación por suma— sólo emergieron al evaluar sobre documentos reales, largos y multilingües. Y dos defectos que habrían costado puntos fueron detectados por las pruebas y el arnés, no por inspección visual.

Los componentes de mayor riesgo regulatorio quedan aislados tras interruptores de configuración, de modo que el sistema puede ajustarse a la interpretación del jurado sin tocar código. Nuestra recomendación, sujeta a esa consulta, es mantener el reranking activo: es la diferencia medida más grande entre una entrada correcta y una competitiva.